



Sri Lanka Institute of Information Technology

Data Warehousing and Business Intelligence IT3021

Assignment 1

2026

Student ID: IT23224384
Student Name: Hasangani WRA
Batch / Group: Y3.S2.DS.WE.01.02
Submitted On: 03/30/2026

Table of Contents

1 TASK 01 - DATASET SELECTION.....	3
1.1 DESCRIPTION	3
2 TASK 02 - PREPARATION OF THE DATA SOURCES	5
3 TASK 03 - SOLUTION ARCHITECTURE	6
4 TASK 04 - DATA WAREHOUSE DESIGN AND DEVELOPMENT	8
5 TASK 05 - ETL DEVELOPMENT.....	16
5.1 EXTRACT DATA FROM SOURCE TO STAGING	16
5.2 TRANSFORMING THE STAGED DATA AND LOADING TO THE DATA WAREHOUSE	22
6 TASK 06 - ETL DEVELOPMENT – ACCUMULATING FACT TABLES.....	34

Task 1 — Dataset Selection

Description

The dataset selected for this assignment is the **Global Superstore Dataset** obtained from Kaggle (<https://www.kaggle.com/datasets/apoorvaappz/global-super-store-dataset>). This is an OLTP dataset containing raw transactional retail sales data covering the period from **2011 to 2014** with **51,290 transaction records**. The dataset represents a global retail store that sells products across multiple categories to customers worldwide. It contains rich transactional data including order details, customer information, product details, shipping information and geographical data making it suitable for building a comprehensive Data Warehouse and Business Intelligence solution. The dataset supports multiple DW/BI requirements including ETL processes, dimensional modeling, SSAS cubes with hierarchies and Power BI reporting.

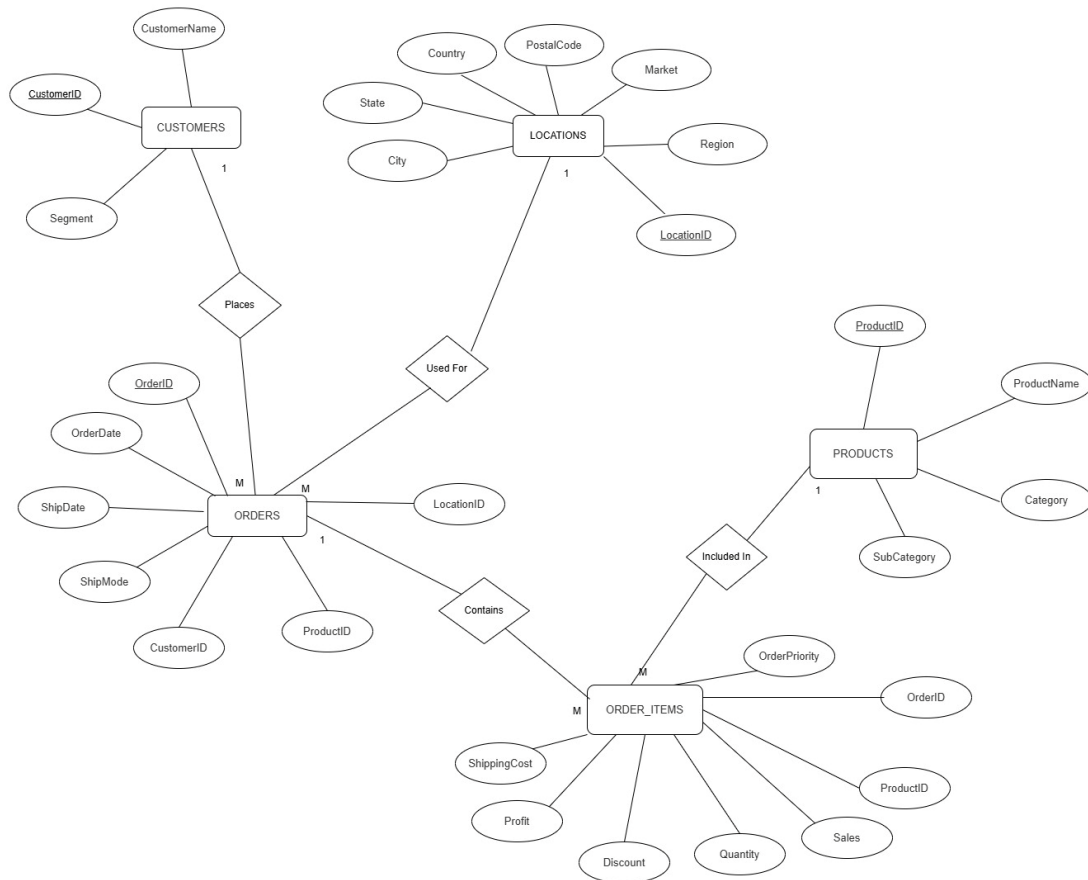
These are data files that I have used,

1. Global_Superstore2.csv
2. Customers.csv
3. Locations.xlsx

ER Diagram

The Global Superstore dataset contains transactional retail data with the following entities:

- ORDERS — core transaction records
- CUSTOMERS — customer information
- PRODUCTS — product details with category hierarchy
- LOCATIONS — geographical information
- ORDER_ITEMS — line items per order with measures



ER-Diagram

Task 2 - Data Source Preparation

The original dataset was split into three different source types to satisfy the multiple source requirement:

Source 1 — SQL Server Database (Superstore_SourceDB): The main transactional file Global_Superstore2.csv was imported into SQL Server as the dbo.Orders table containing 51,290 rows with 24 columns including order details, product information, customer data and financial measures.

Source 2 — CSV Flat File (Customers.csv): Customer information was extracted from the main dataset and saved as a separate CSV flat file containing 1,590 unique customers with columns: Customer ID, Customer Name and Segment.

Source 3 — Excel File (Locations.xlsx): Geographical location data was extracted and saved as an Excel file containing 3,847 unique locations with columns: City, State, Country, Postal Code, Market and Region.

 Customers.csv	3/25/2026 1:13 PM	Microsoft Excel Com...	52 KB
 Global_Superstore2.csv	3/25/2026 4:21 AM	Microsoft Excel Com...	11,752 KB
 Locations.xlsx	3/25/2026 1:18 PM	Microsoft Excel Work...	149 KB

Task 3 — Solution Architecture

The following high-level DW/BI solution architecture was designed for the Global Superstore dataset. The architecture consists of five main layers:

1. Data Source Layer: Contains three different source types — SQL Server database (orders), CSV flat file (customers) and Excel file (locations).

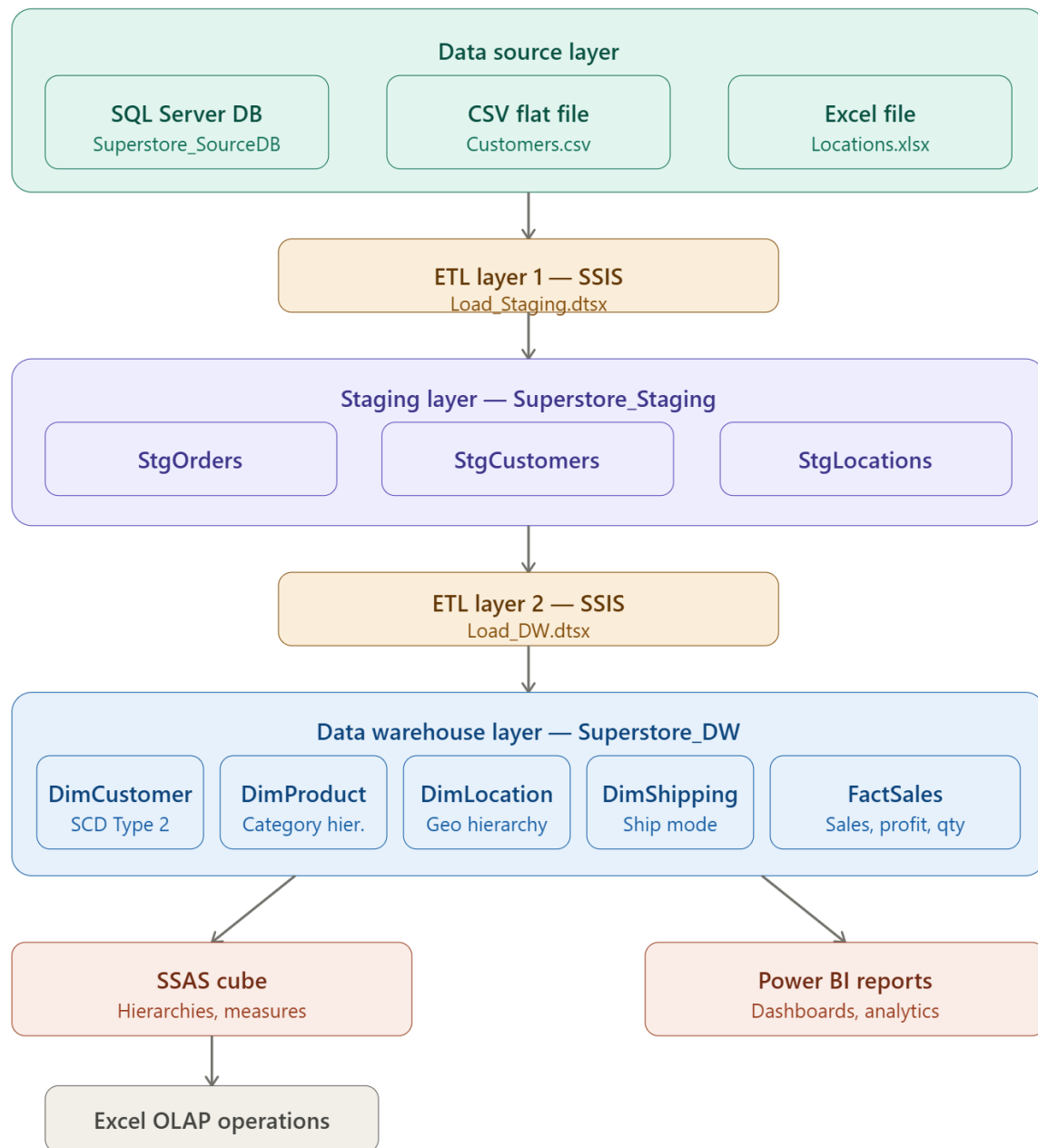
2. ETL Layer 1 (SSIS — Load_Staging.dtsx): SQL Server Integration Services is used to extract data from all three sources and load them into the staging database. Each source has a dedicated Data Flow Task with OnPreExecute event handlers to truncate staging tables before loading.

3. Staging Layer (Superstore_Staging): An intermediate database that temporarily stores data from all sources in a unified location before transformation. Contains StgOrders, StgCustomers and StgLocations tables.

4. ETL Layer 2 (SSIS — Load_DW.dtsx): A second SSIS package extracts data from staging, performs transformations and loads into the data warehouse using stored procedures.

5. Data Warehouse Layer (Superstore_DW): The final dimensional model containing dimension tables and fact table in a Star Schema.

6. BI Layer: SSAS Cube for multidimensional analysis and Power BI for reporting and dashboards.



Solution Architecture Diagram

Task 4 — DW Design

Schema Used: Star Schema

A Star Schema was designed for the Global Superstore Data Warehouse. The schema consists of one central fact table connected to five dimension tables.

Dimension Tables:

- **DimCustomer** — Contains customer details including CustomerSK (surrogate key), AlternateCustomerID, CustomerName, Segment, StartDate, EndDate and IsCurrent. This is implemented as a Slowly Changing Dimension (SCD Type 2) to track historical changes in customer segments.
- **DimProduct** — Contains product information including ProductSK (surrogate key), AlternateProductID, ProductName, Category and SubCategory. Natural hierarchy exists: Category → SubCategory → Product.
- **DimLocation** — Contains geographical data including LocationSK (surrogate key), City, State, Country, PostalCode, Market and Region. Natural hierarchy exists: Country → State → City.
- **DimShipping** — Contains shipping mode information with ShippingSK (surrogate key) and ShipMode.
- **DimDate** — Common date dimension containing DateKey, FullDate, Day, Month, MonthName, Quarter and Year. Generated for years 2011-2015 with 1,826 date records.

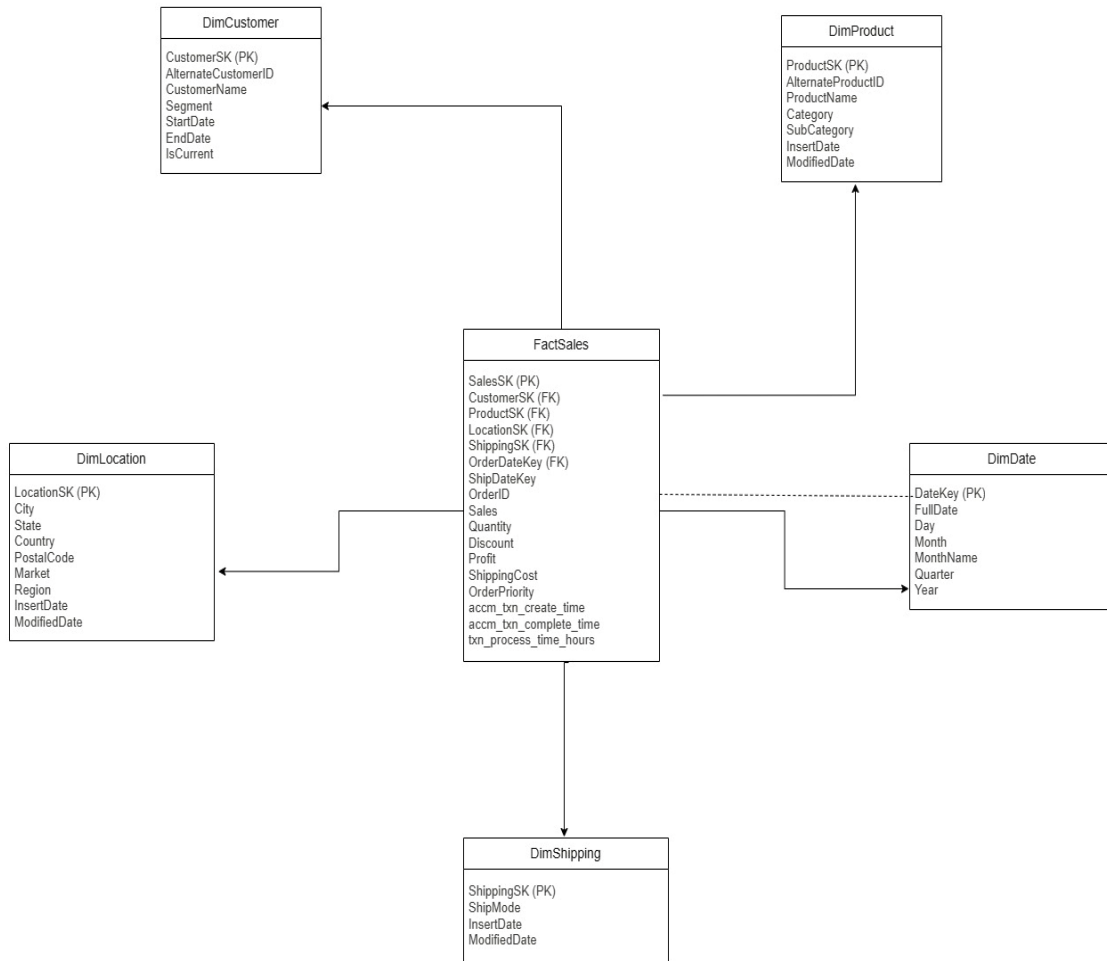
Fact Table:

- **FactSales** — Central fact table containing measures: Sales, Quantity, Discount, Profit, ShippingCost and foreign keys to all dimension tables. Also contains accumulating fact columns: accm_txn_create_time, accm_txn_complete_time and txn_process_time_hours (computed column).

Assumptions:

- Product, Location and Shipping data does not change over time so SCD Type 1 approach is used.
- Customer Segment may change over time so SCD Type 2 is implemented on DimCustomer.
- Date keys are stored as integers in YYYYMMDD format.

Dimension Diagram



Dimension Model

Query 1 — Create Databases

sql

```
CREATE DATABASE Superstore_SourceDB;  
CREATE DATABASE Superstore_Staging;  
CREATE DATABASE Superstore_DW;
```

Description: Three databases were created — source database, staging database and data warehouse database.

Query 2 — Create Staging Tables

sql

```
USE Superstore_Staging;  
  
CREATE TABLE StgOrders (  
    RowID INT,  
    OrderID NVARCHAR(50),  
    OrderDate NVARCHAR(50),  
    ShipDate NVARCHAR(50),  
    ShipMode NVARCHAR(50),  
    CustomerID NVARCHAR(50),  
    CustomerName NVARCHAR(100),  
    Segment NVARCHAR(50),  
    City NVARCHAR(100),  
    State NVARCHAR(100),  
    Country NVARCHAR(100),  
    PostalCode NVARCHAR(50),  
    Market NVARCHAR(50),  
    Region NVARCHAR(50),  
    ProductID NVARCHAR(50),  
    Category NVARCHAR(50),
```

```
SubCategory NVARCHAR(50),
ProductName NVARCHAR(255),
Sales NVARCHAR(50),
Quantity NVARCHAR(50),
Discount NVARCHAR(50),
Profit NVARCHAR(50),
ShippingCost NVARCHAR(50),
OrderPriority NVARCHAR(50)
);
```

```
CREATE TABLE StgCustomers (
    CustomerID NVARCHAR(50),
    CustomerName NVARCHAR(100),
    Segment NVARCHAR(50)
);
```

```
CREATE TABLE StgLocations (
    City NVARCHAR(100),
    State NVARCHAR(100),
    Country NVARCHAR(100),
    PostalCode NVARCHAR(50),
    Market NVARCHAR(50),
    Region NVARCHAR(50)
);
```

Description: Three staging tables were created in the Superstore_Staging database to temporarily store data from all three source types before loading into the data warehouse.

Query 3 — Create DW Tables

sql

USE Superstore_DW;

```
CREATE TABLE DimDate (  
    DateKey INT PRIMARY KEY,  
    FullDate DATE,  
    Day INT,  
    Month INT,  
    MonthName NVARCHAR(20),  
    Quarter INT,  
    Year INT  
);
```

```
CREATE TABLE DimCustomer (  
    CustomerSK INT IDENTITY(1,1) PRIMARY KEY,  
    AlternateCustomerID NVARCHAR(50),  
    CustomerName NVARCHAR(100),  
    Segment NVARCHAR(50),  
    StartDate DATETIME,  
    EndDate DATETIME,  
    IsCurrent BIT  
);
```

```
CREATE TABLE DimProduct (  
    ProductSK INT IDENTITY(1,1) PRIMARY KEY,  
    AlternateProductID NVARCHAR(50),  
    ProductName NVARCHAR(255),  
    Category NVARCHAR(50),  
    SubCategory NVARCHAR(50),
```

```
    InsertDate DATETIME,  
    ModifiedDate DATETIME  
);
```

```
CREATE TABLE DimLocation (  
    LocationSK INT IDENTITY(1,1) PRIMARY KEY,  
    City NVARCHAR(100),  
    State NVARCHAR(100),  
    Country NVARCHAR(100),  
    PostalCode NVARCHAR(50),  
    Market NVARCHAR(50),  
    Region NVARCHAR(50),  
    InsertDate DATETIME,  
    ModifiedDate DATETIME  
);
```

```
CREATE TABLE DimShipping (  
    ShippingSK INT IDENTITY(1,1) PRIMARY KEY,  
    ShipMode NVARCHAR(50),  
    InsertDate DATETIME,  
    ModifiedDate DATETIME  
);
```

```
CREATE TABLE FactSales (  
    SalesSK INT IDENTITY(1,1) PRIMARY KEY,  
    CustomerSK INT FOREIGN KEY REFERENCES DimCustomer(CustomerSK),  
    ProductSK INT FOREIGN KEY REFERENCES DimProduct(ProductSK),  
    LocationSK INT FOREIGN KEY REFERENCES DimLocation(LocationSK),  
    ShippingSK INT FOREIGN KEY REFERENCES DimShipping(ShippingSK),  
    OrderDateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),
```

```

ShipDateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),
OrderID NVARCHAR(50),
Sales DECIMAL(18,2),
Quantity INT,
Discount DECIMAL(18,2),
Profit DECIMAL(18,2),
ShippingCost DECIMAL(18,2),
OrderPriority NVARCHAR(50),
accm_txn_create_time DATETIME,
accm_txn_complete_time DATETIME,
txn_process_time_hours AS (
    CAST(DATEDIFF(MINUTE, accm_txn_create_time,
        accm_txn_complete_time) AS DECIMAL(18,2)) / 60
)
);

```

Description: All dimension tables and the fact table were created in the Superstore_DW database following the Star Schema dimensional model.

Query 4 — Generate Date Dimension

sql

```
USE Superstore_DW;
```

```
DECLARE @StartDate DATE = '2011-01-01'
```

```
DECLARE @EndDate DATE = '2015-12-31'
```

```
WHILE @StartDate <= @EndDate
```

```
BEGIN
```

```
    INSERT INTO DimDate
```

```
        (DateKey, FullDate, Day, Month, MonthName, Quarter, Year)
```

```
VALUES (  
    CONVERT(INT, CONVERT(VARCHAR, @StartDate, 112)),  
    @StartDate,  
    DAY(@StartDate),  
    MONTH(@StartDate),  
    DATENAME(MONTH, @StartDate),  
    DATEPART(QUARTER, @StartDate),  
    YEAR(@StartDate)  
)  
SET @StartDate = DATEADD(DAY, 1, @StartDate)  
END  
  
SELECT COUNT(*) AS TotalDates FROM DimDate;
```

Description: The date dimension was populated with 1,826 date records covering the period from 2011 to 2015 using a WHILE loop.

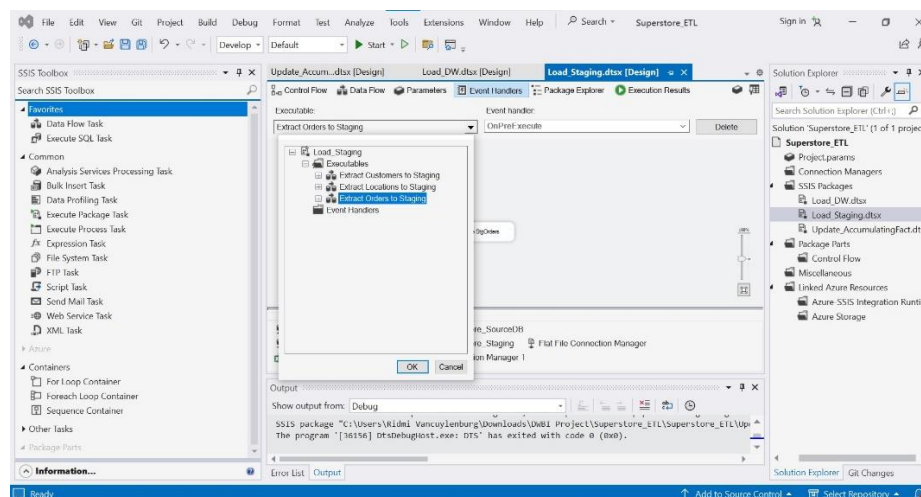
Task 5 — ETL Development

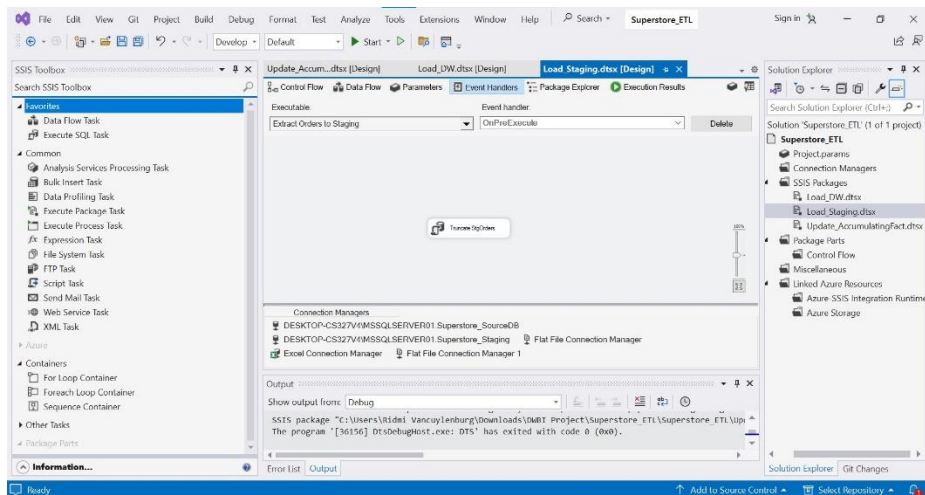
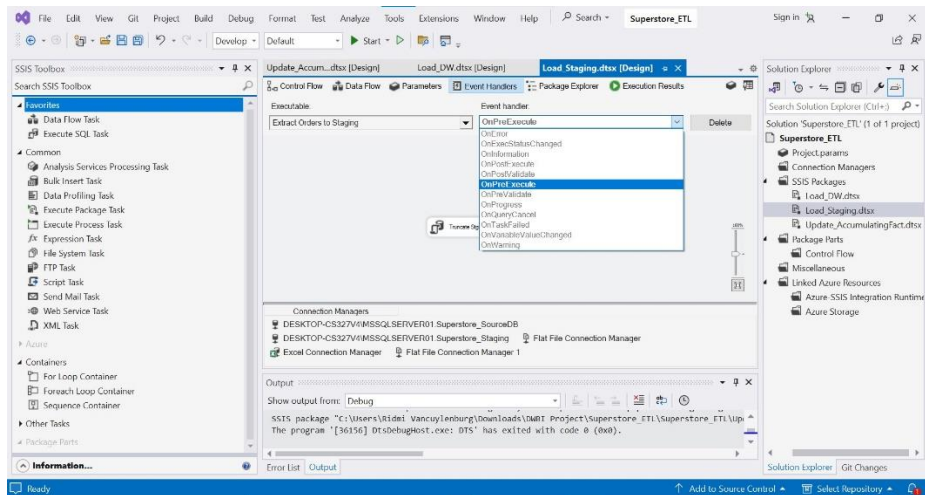
Package 1: Load_Staging.dtsx (Source to Staging)

This package extracts data from all three source types and loads them into the staging database. Three Data Flow Tasks are connected serially:

1. **Extract Orders to Staging** — Extracts 51,290 rows from the SQL Server source database (Superstore_SourceDB.Orders) and loads into StgOrders staging table.

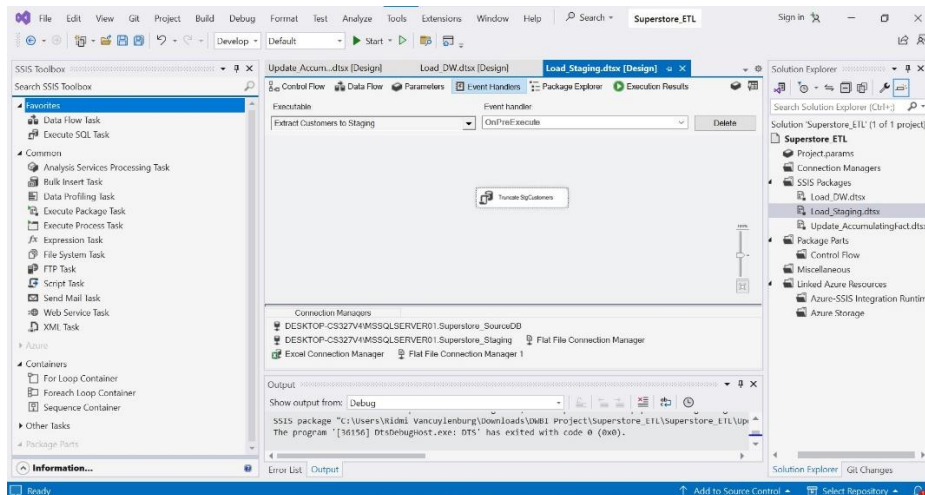
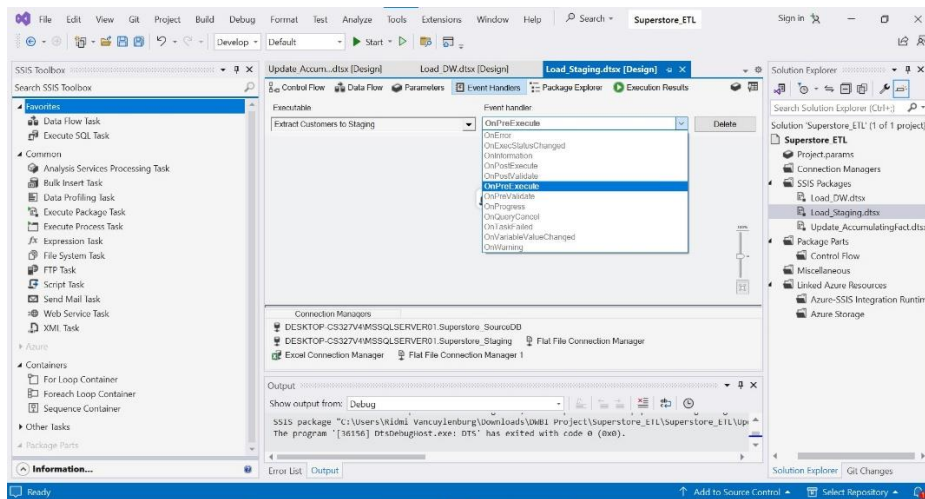
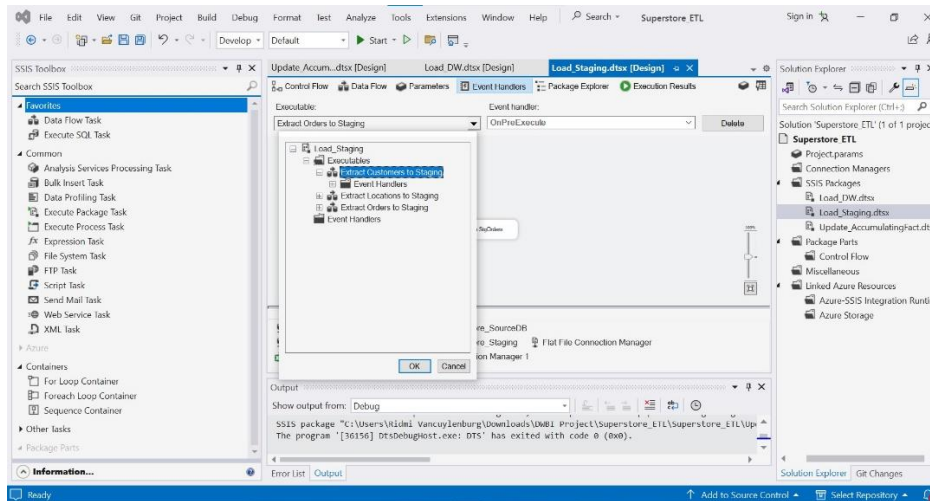
An OnPreExecute event handler was configured for the Extract Orders to Staging data flow task. This event handler contains an Execute SQL Task that truncates the StgOrders staging table before each execution to prevent duplicate data from being loaded.





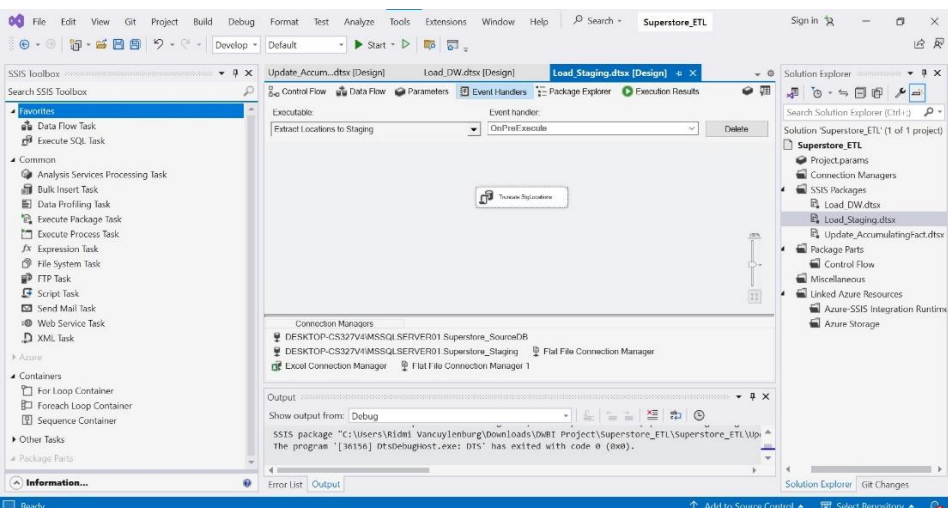
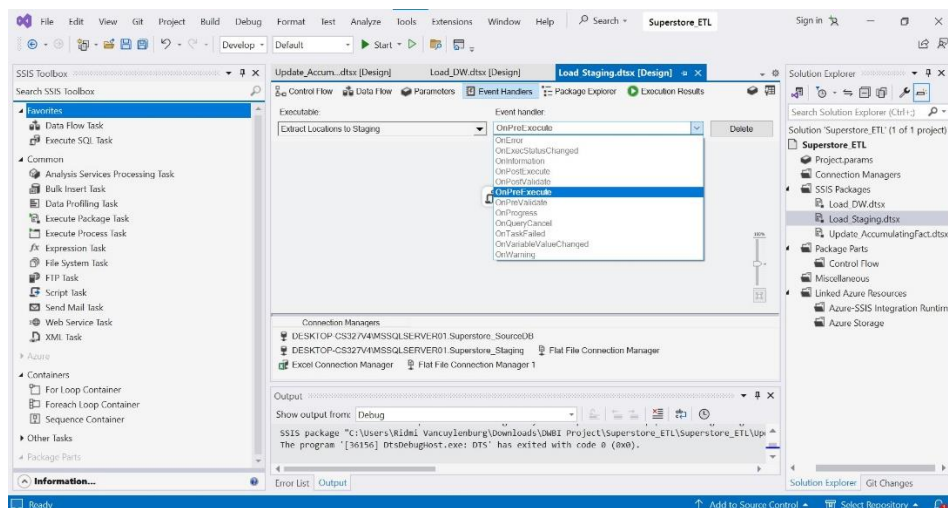
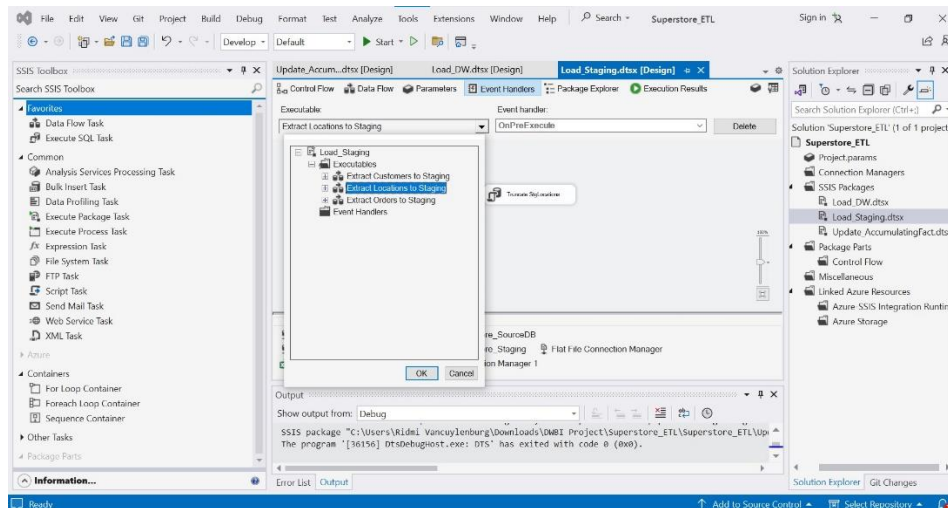
2. **Extract Customers to Staging** — Extracts 1,590 unique customer records from Customers.csv flat file and loads into StgCustomers staging table.

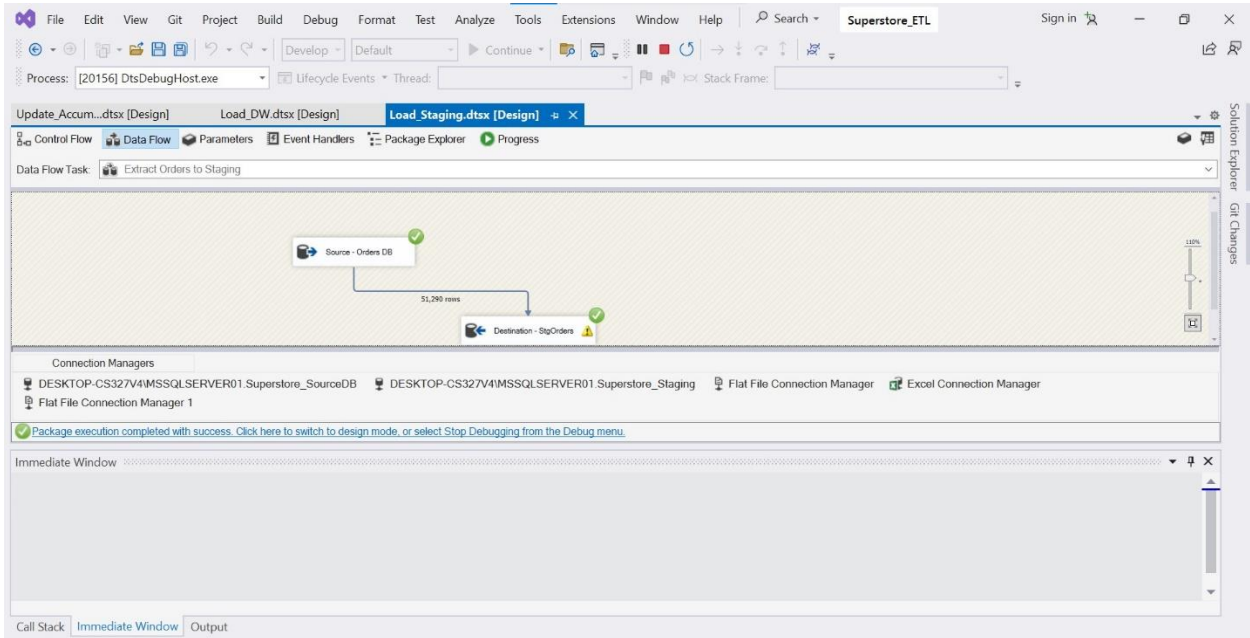
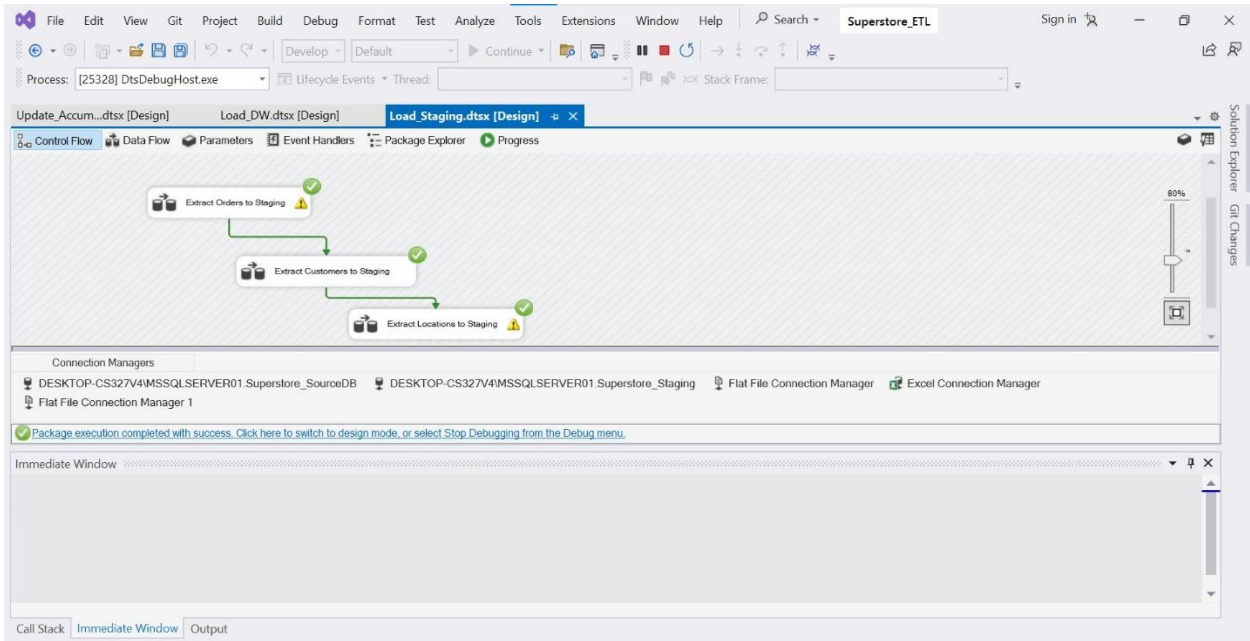
An OnPreExecute event handler was configured for the Extract Customers to Staging data flow task. The Execute SQL Task truncates the StgCustomers table before each execution.

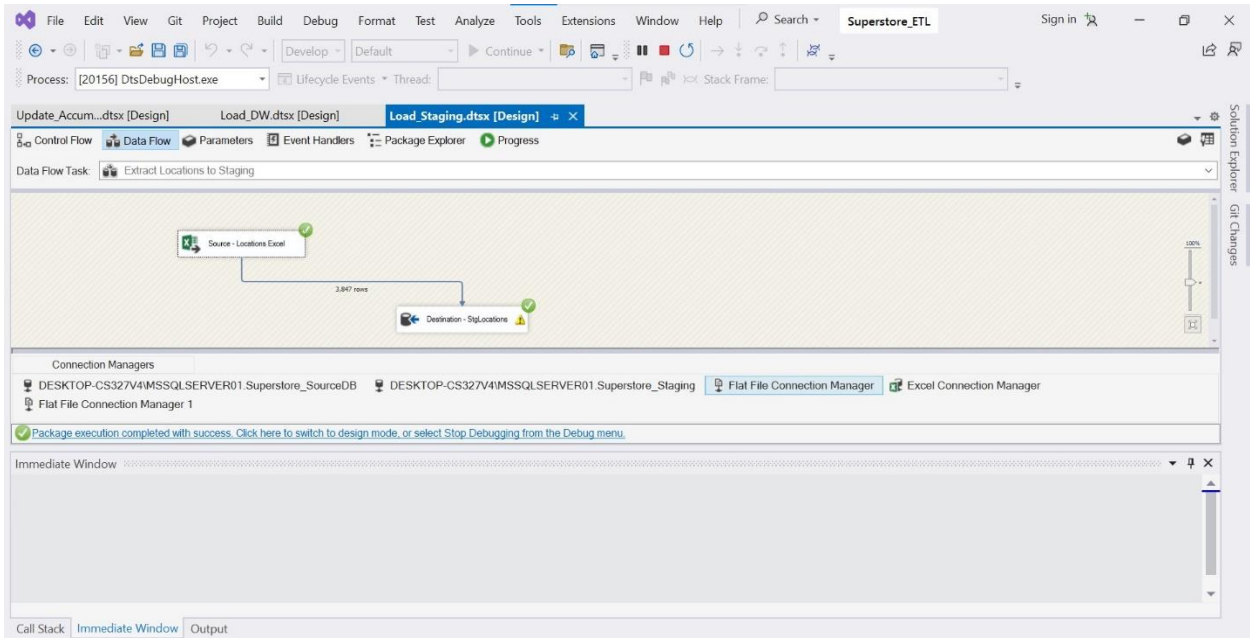
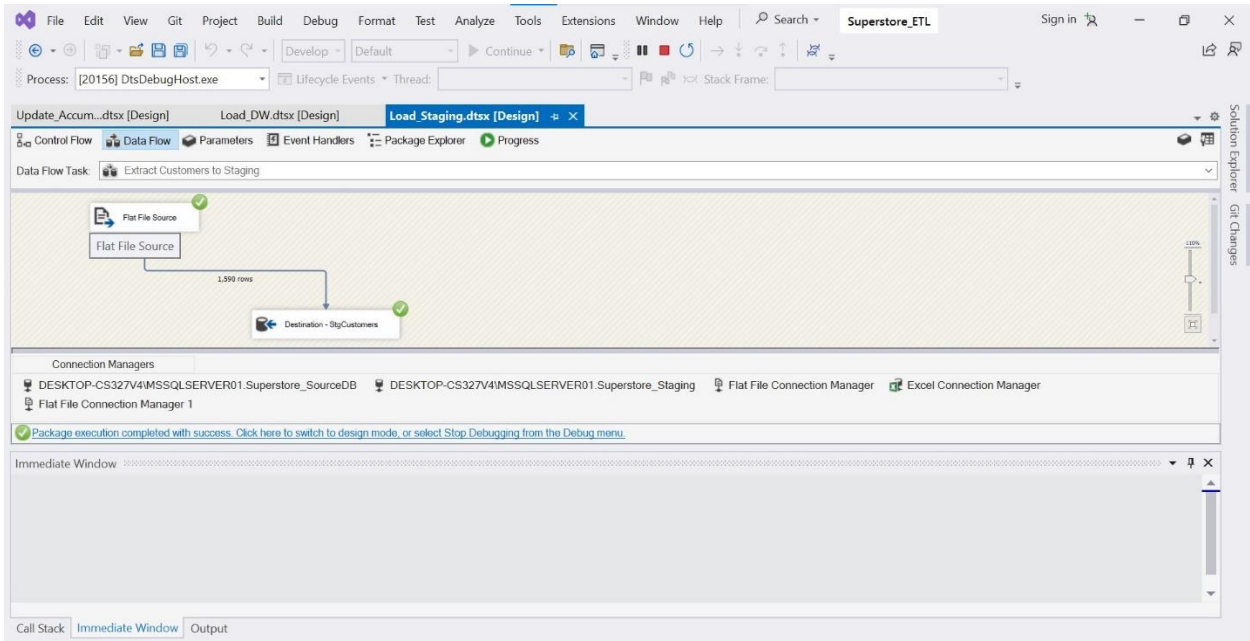


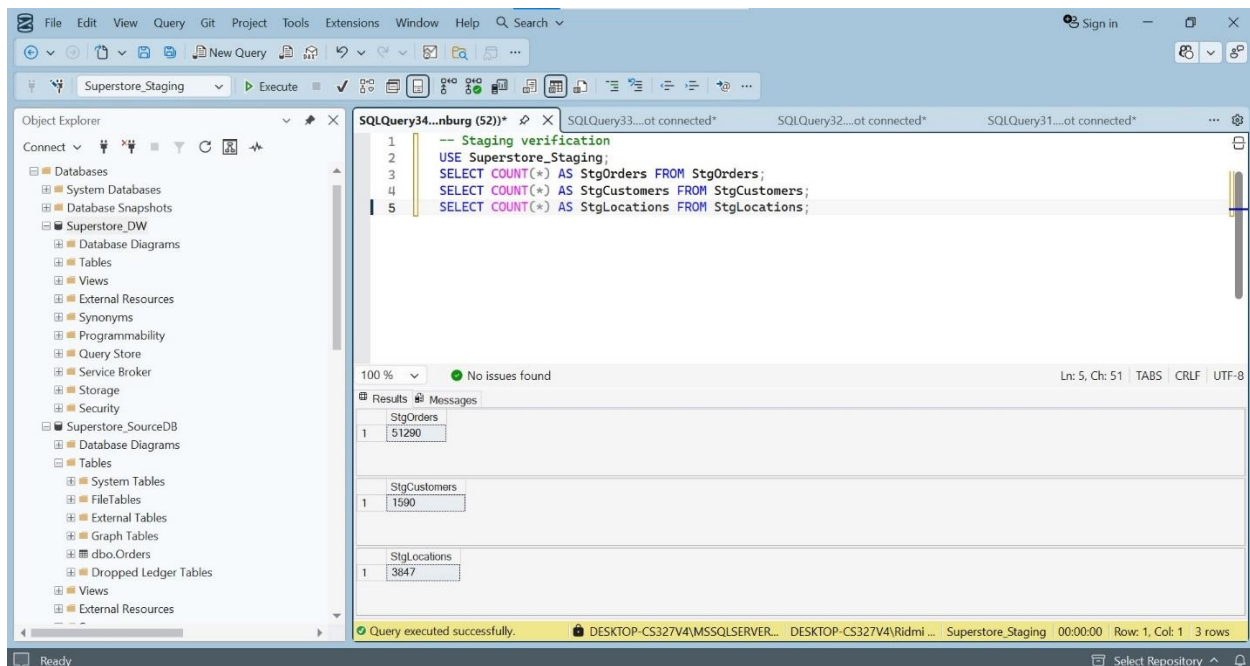
3. Extract Locations to Staging — Extracts 3,847 unique location records from Locations.xlsx Excel file and loads into StgLocations staging table.

An OnPreExecute event handler was configured for the Extract Locations to Staging data flow task. The Execute SQL Task truncates the StgLocations table before each execution.





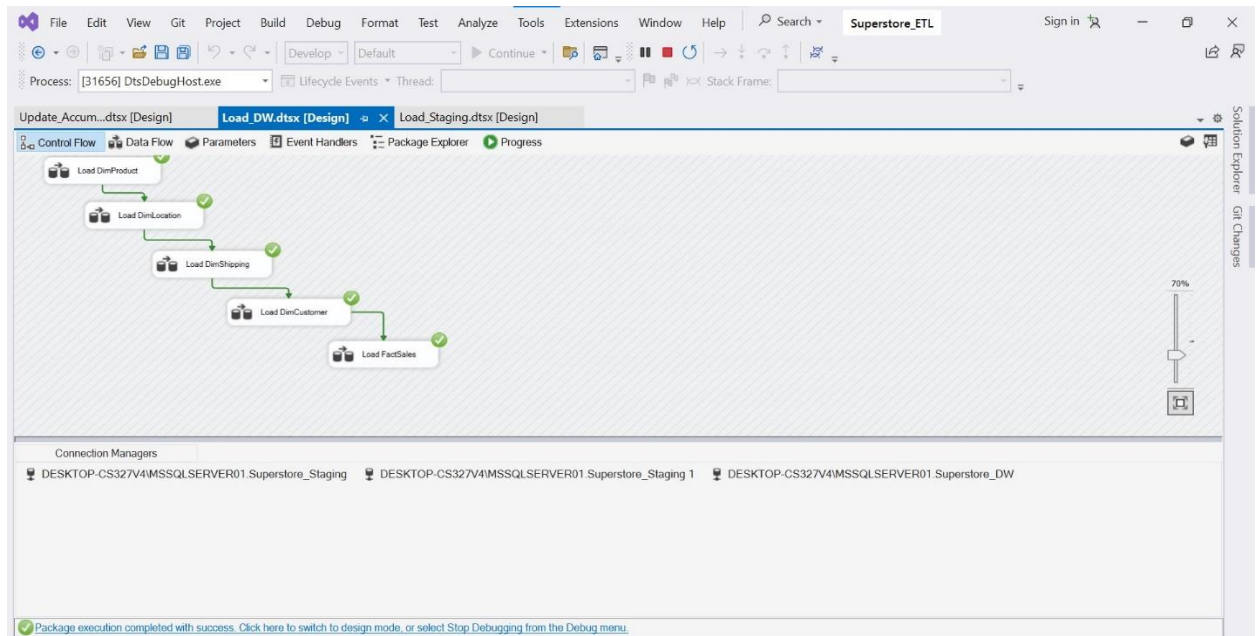
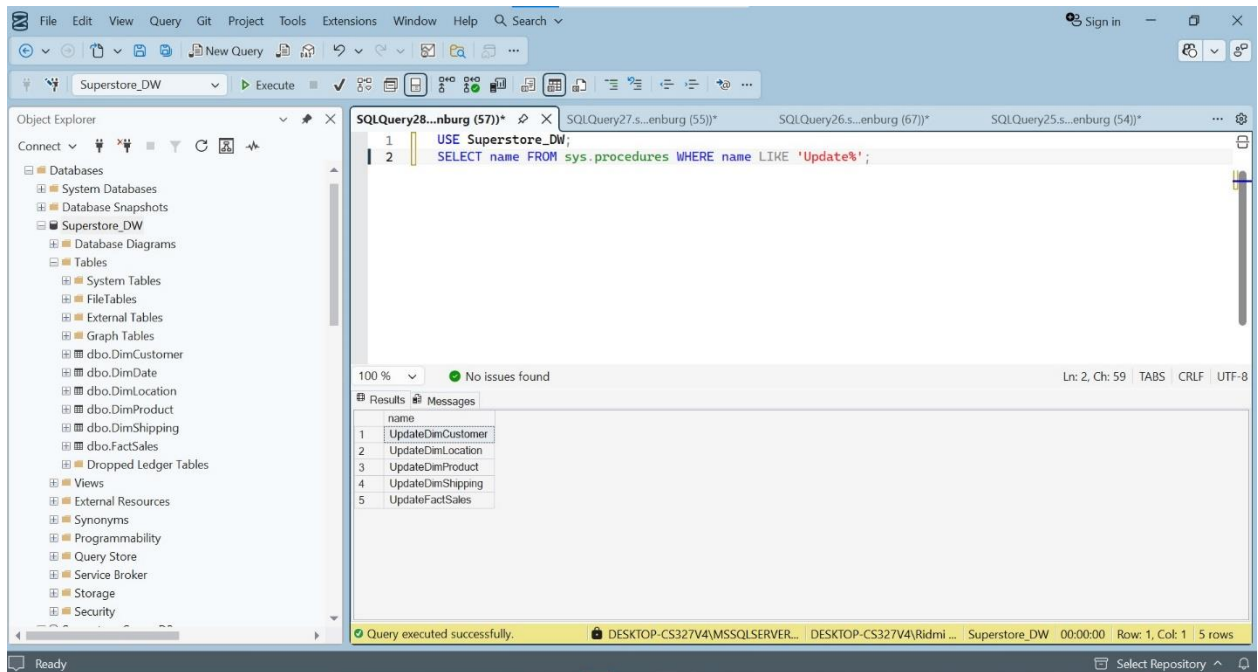


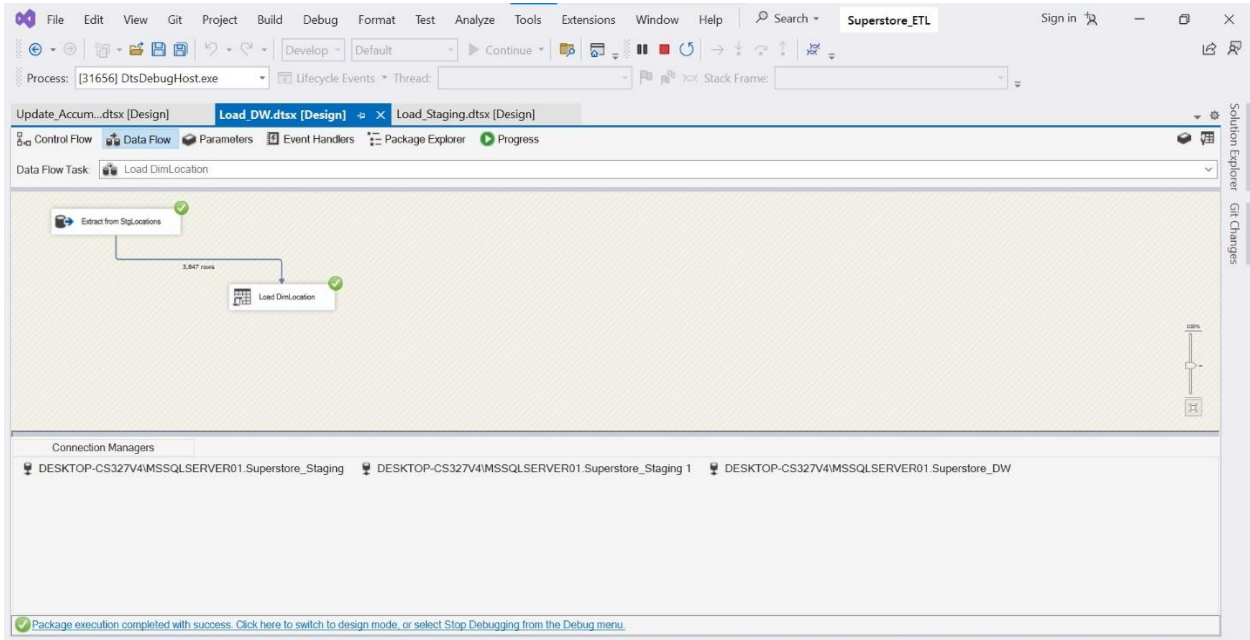
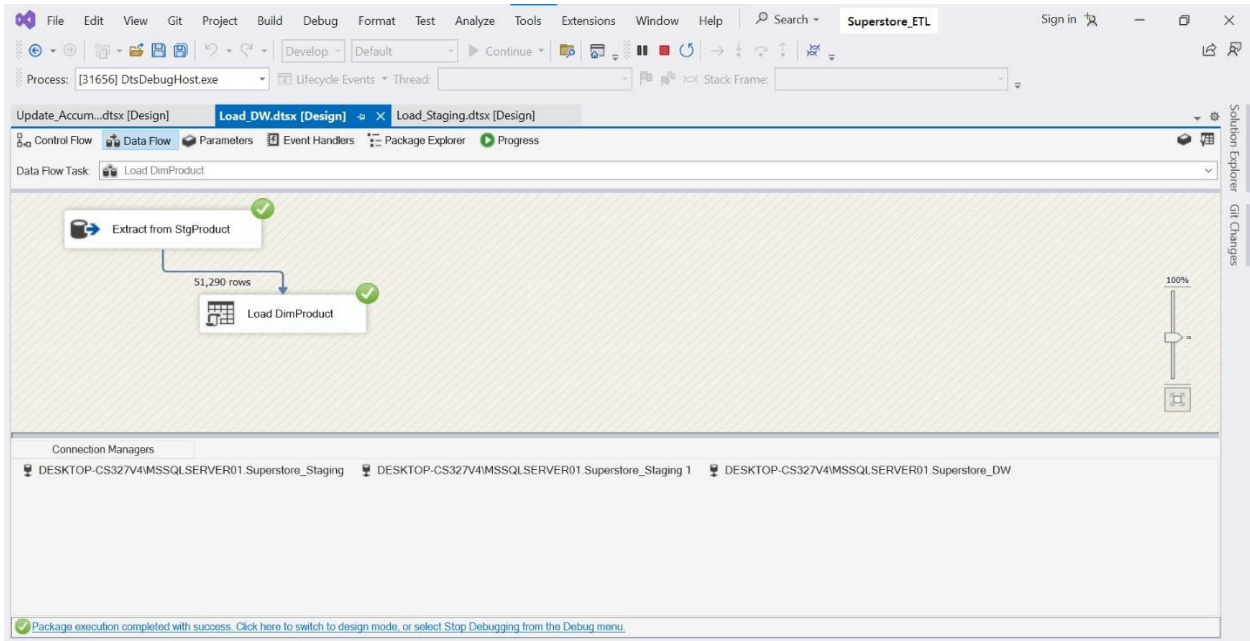


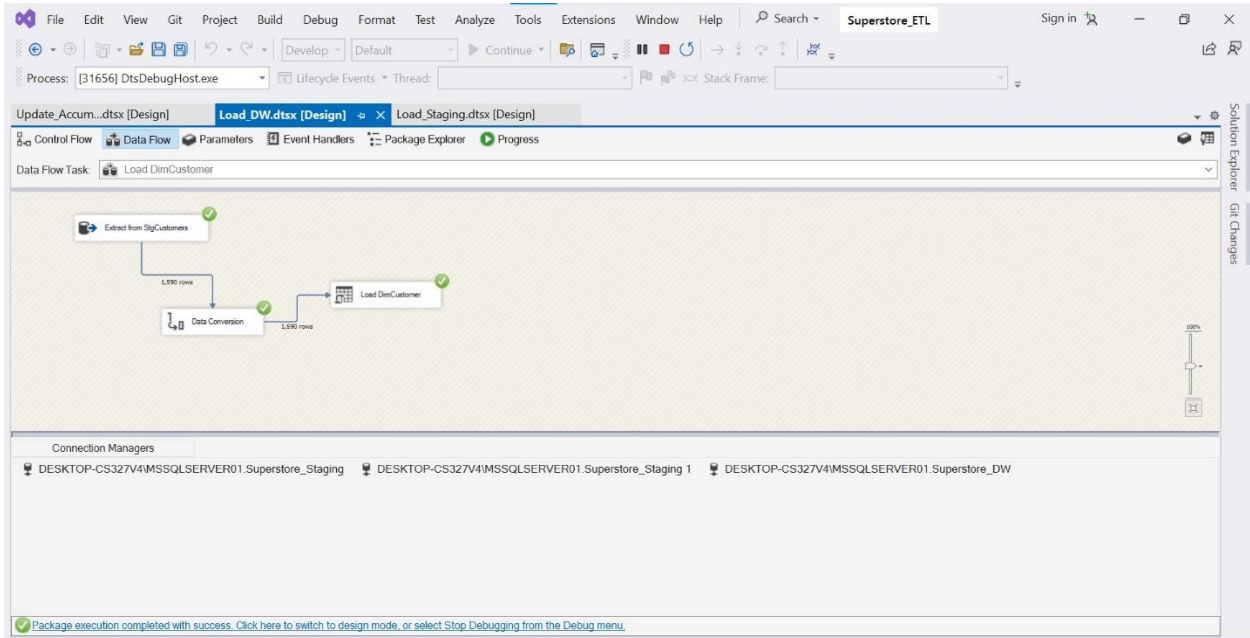
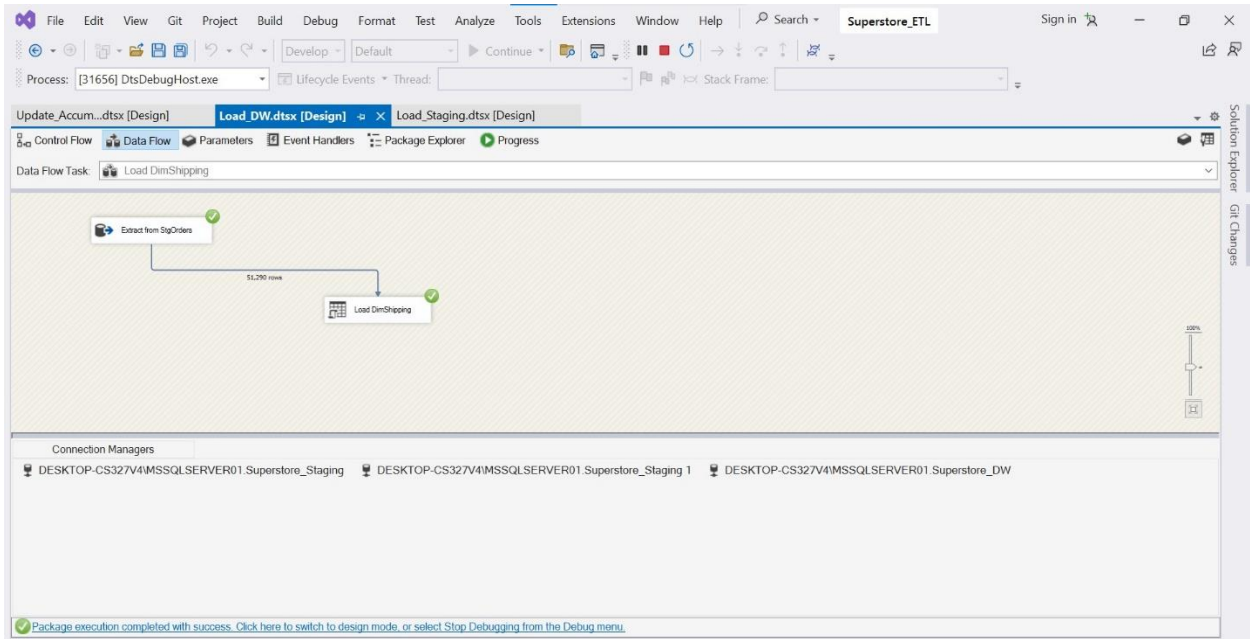
Package 2: Load_DW.dtsx (Staging to Data Warehouse)

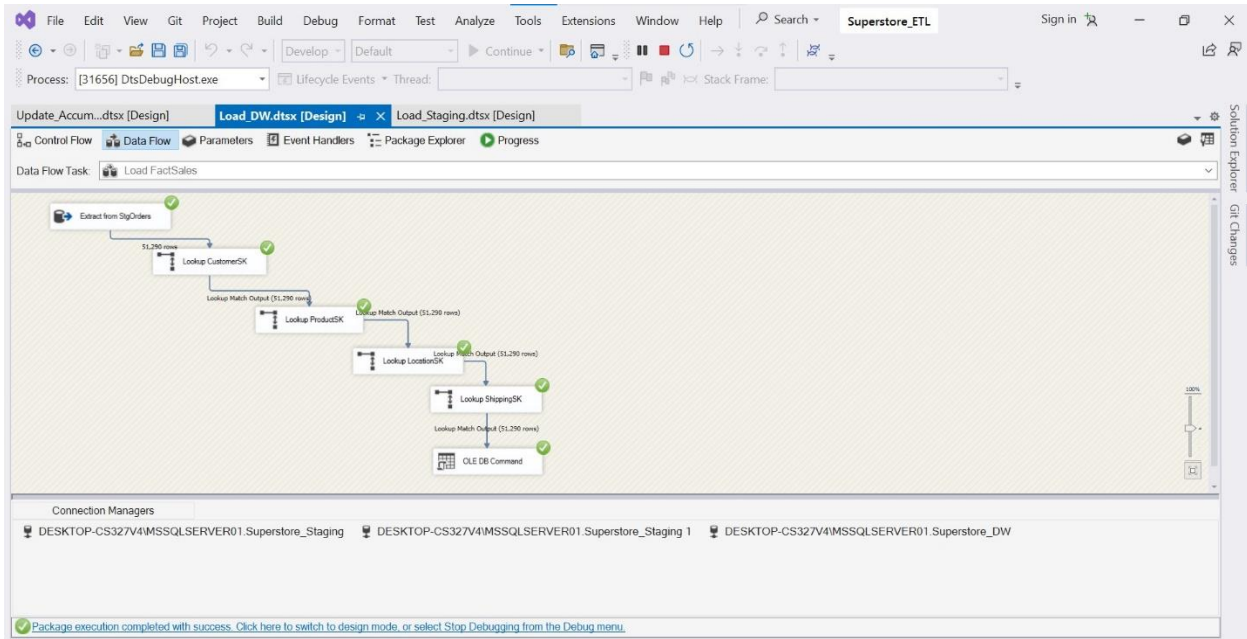
This package transforms and loads data from staging into the data warehouse dimensional model. Five Data Flow Tasks are connected serially in the correct order to respect foreign key relationships:

1. **Load DimProduct** — Extracts from StgOrders and loads into DimProduct using UpdateDimProduct stored procedure (upsert logic).
2. **Load DimLocation** — Extracts from StgLocations and loads into DimLocation using UpdateDimLocation stored procedure.
3. **Load DimShipping** — Extracts from StgOrders and loads into DimShipping using UpdateDimShipping stored procedure.
4. **Load DimCustomer** — Extracts from StgCustomers and loads into DimCustomer using UpdateDimCustomer stored procedure with SCD Type 2 logic.
5. **Load FactSales** — Most complex task — uses four Lookup transformations to retrieve surrogate keys (CustomerSK, ProductSK, LocationSK, ShippingSK) from dimension tables, then loads into FactSales using UpdateFactSales stored procedure.









Visual Studio Code interface showing the SQL query execution results for 'SQLQuery33...nburg (90)*'.

Query:

```
USE Superstore_DW;
SELECT OrderID, accm_txn_create_time, accm_txn_complete_time, txn_process_time_hours
FROM FactSales
WHERE OrderID IN (
    'IT-2012-3490826',
    'MX-2011-123911',
    'MO-2014-6730',
    'IN-2012-76401',
    'TU-2014-6290'
);
```

Results:

OrderID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
IT-2012-3490826	2026-03-29 16:17:38.430	2012-03-15 12:00:00.000	-123052.283333
MX-2011-123911	2026-03-29 16:17:38.437	2011-06-20 12:00:00.000	-129508.283333
MO-2014-6730	2026-03-29 16:17:38.437	2014-05-10 12:00:00.000	-104188.283333
IN-2012-76401	2026-03-29 16:17:38.437	2012-08-25 12:00:00.000	-119140.283333
TU-2014-6290	2026-03-29 16:25:55.327	2014-11-30 12:00:00.000	-99292.416666

Query executed successfully. DESKTOP-CS327V4\MSSQLSERVER... DESKTOP-CS327V4\Ridmi... Superstore_DW 00:00:00 Row: 1, Col: 1 5 rows

Query 5 — UpdateDimProduct Stored Procedure

sql

USE Superstore_DW;

GO

CREATE PROCEDURE dbo.UpdateDimProduct

 @ProductID NVARCHAR(50),

 @ProductName NVARCHAR(255),

 @Category NVARCHAR(50),

 @SubCategory NVARCHAR(50)

AS

BEGIN

 IF NOT EXISTS (SELECT ProductSK FROM DimProduct

 WHERE AlternateProductID = @ProductID)

 INSERT INTO DimProduct

 (AlternateProductID, ProductName, Category, SubCategory,

 InsertDate, ModifiedDate)

 VALUES (@ProductID, @ProductName, @Category,

 @SubCategory, GETDATE(), GETDATE())

 ELSE

 UPDATE DimProduct

 SET ProductName = @ProductName,

 Category = @Category,

 SubCategory = @SubCategory,

 ModifiedDate = GETDATE()

 WHERE AlternateProductID = @ProductID

END;

GO

Description: This stored procedure implements upsert logic for DimProduct. If a product already exists it updates the record, otherwise it inserts a new record.

Query 6 — UpdateDimLocation Stored Procedure

sql

USE Superstore_DW;

GO

CREATE PROCEDURE dbo.UpdateDimLocation

 @City NVARCHAR(100),

 @State NVARCHAR(100),

 @Country NVARCHAR(100),

 @PostalCode NVARCHAR(50),

 @Market NVARCHAR(50),

 @Region NVARCHAR(50)

AS

BEGIN

 IF NOT EXISTS (SELECT LocationSK FROM DimLocation

 WHERE City = @City

 AND State = @State

 AND Country = @Country)

 INSERT INTO DimLocation

 (City, State, Country, PostalCode, Market, Region,

 InsertDate, ModifiedDate)

 VALUES (@City, @State, @Country, @PostalCode,

 @Market, @Region, GETDATE(), GETDATE())

ELSE

 UPDATE DimLocation

 SET PostalCode = @PostalCode,

 Market = @Market,

 Region = @Region,

 ModifiedDate = GETDATE()

 WHERE City = @City

```
        AND State = @State
        AND Country = @Country
END;
GO
```

Description: This stored procedure implements upsert logic for DimLocation based on the combination of City, State and Country as the natural key.

Query 7 — UpdateDimShipping Stored Procedure

```
sql
USE Superstore_DW;
GO

CREATE PROCEDURE dbo.UpdateDimShipping
    @ShipMode NVARCHAR(50)
AS
BEGIN
    IF NOT EXISTS (SELECT ShippingSK FROM DimShipping
        WHERE ShipMode = @ShipMode)
    INSERT INTO DimShipping
        (ShipMode, InsertDate, ModifiedDate)
    VALUES (@ShipMode, GETDATE(), GETDATE())
END;
GO
```

Description: This stored procedure inserts new shipping modes into DimShipping only if they do not already exist.

Query 8 — UpdateDimCustomer Stored Procedure (SCD Type 2)

sql

USE Superstore_DW;

GO

CREATE PROCEDURE dbo.UpdateDimCustomer

 @CustomerID NVARCHAR(50),

 @CustomerName NVARCHAR(100),

 @Segment NVARCHAR(50)

AS

BEGIN

 IF NOT EXISTS (SELECT CustomerSK FROM DimCustomer

 WHERE AlternateCustomerID = @CustomerID

 AND IsCurrent = 1)

 BEGIN

 INSERT INTO DimCustomer

 (AlternateCustomerID, CustomerName, Segment,

 StartDate, EndDate, IsCurrent)

 VALUES (@CustomerID, @CustomerName, @Segment,

 GETDATE(), NULL, 1)

 END

 ELSE

 BEGIN

 IF EXISTS (SELECT CustomerSK FROM DimCustomer

 WHERE AlternateCustomerID = @CustomerID

 AND IsCurrent = 1

 AND Segment <> @Segment)

 BEGIN

 UPDATE DimCustomer

 SET EndDate = GETDATE(), IsCurrent = 0

 WHERE AlternateCustomerID = @CustomerID

```

        AND IsCurrent = 1

INSERT INTO DimCustomer
(AlternateCustomerID, CustomerName, Segment,
StartDate, EndDate, IsCurrent)
VALUES (@CustomerID, @CustomerName, @Segment,
        GETDATE(), NULL, 1)

END

END

END;

GO

```

Description: This stored procedure implements SCD Type 2 logic for DimCustomer. When a customer's segment changes, the existing record is expired and a new record is inserted to maintain full history.

Query 9 — UpdateFactSales Stored Procedure

sql

```

USE Superstore_DW;

GO

```

```

CREATE PROCEDURE dbo.UpdateFactSales
    @OrderID NVARCHAR(50),
    @CustomerSK INT,
    @ProductSK INT,
    @LocationSK INT,
    @ShippingSK INT,
    @OrderDateKey NVARCHAR(50),
    @ShipDateKey NVARCHAR(50),
    @Sales NVARCHAR(50),
    @Quantity NVARCHAR(50),
    @Discount NVARCHAR(50),
    @Profit NVARCHAR(50),

```

```

@ShippingCost NVARCHAR(50),
@OrderPriority NVARCHAR(50)
AS
BEGIN
    DECLARE @OrderDateInt INT
    DECLARE @ShipDateInt INT

    SET @OrderDateInt = TRY_CAST(
        FORMAT(TRY_CAST(@OrderDateKey AS DATE), 'yyyyMMdd') AS INT)
    SET @ShipDateInt = TRY_CAST(
        FORMAT(TRY_CAST(@ShipDateKey AS DATE), 'yyyyMMdd') AS INT)

    IF NOT EXISTS (SELECT SalesSK FROM FactSales
        WHERE OrderID = @OrderID)
    BEGIN
        INSERT INTO FactSales
        (OrderID, CustomerSK, ProductSK, LocationSK, ShippingSK,
        OrderDateKey, ShipDateKey, Sales, Quantity, Discount,
        Profit, ShippingCost, OrderPriority, accm_txn_create_time)
        VALUES
        (@OrderID, @CustomerSK, @ProductSK, @LocationSK, @ShippingSK,
        @OrderDateInt, @ShipDateInt,
        TRY_CAST(@Sales AS DECIMAL(18,2)),
        TRY_CAST(@Quantity AS INT),
        TRY_CAST(@Discount AS DECIMAL(18,2)),
        TRY_CAST(@Profit AS DECIMAL(18,2)),
        TRY_CAST(@ShippingCost AS DECIMAL(18,2)),
        @OrderPriority, GETDATE())
    END
END;
GO

```


Description: This stored procedure inserts new sales records into FactSales with all surrogate keys, measures and the accm_txn_create_time set to the current system time.

Query 10 — Verify Staging Data

sql

```
USE Superstore_Staging;
```

```
SELECT COUNT(*) AS StgOrders FROM StgOrders;
```

```
SELECT COUNT(*) AS StgCustomers FROM StgCustomers;
```

```
SELECT COUNT(*) AS StgLocations FROM StgLocations;
```

Description: These queries verify that data was successfully loaded into all three staging tables.

Query 11 — Verify DW Data

sql

```
USE Superstore_DW;
```

```
SELECT COUNT(*) AS Products FROM DimProduct;
```

```
SELECT COUNT(*) AS Locations FROM DimLocation;
```

```
SELECT COUNT(*) AS Shippings FROM DimShipping;
```

```
SELECT COUNT(*) AS Customers FROM DimCustomer;
```

```
SELECT COUNT(*) AS Sales FROM FactSales;
```

Description: These queries verify that data was successfully loaded into all dimension tables and the fact table in the data warehouse.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with the 'Superstore_DW' database selected. The right pane shows a query window with the following SQL code:

```
1  -- Show current records
2  USE Superstore_DW;
3  SELECT TOP 5
4      CustomerSK,
5      AlternateCustomerID,
6      CustomerName,
7      Segment,
8      StartDate,
9      EndDate,
10     IsCurrent
11 FROM DimCustomer
12 ORDER BY AlternateCustomerID;
13
14 -- Expire old record
```

The query results are displayed in a table with the following columns: CustomerSK, AlternateCustomerID, CustomerName, Segment, StartDate, EndDate, and IsCurrent. The results show 5 rows of data.

CustomerSK	AlternateCustomerID	CustomerName	Segment	StartDate	EndDate	IsCurrent
487	AA-10315	Alex Avila	Consumer	2026-03-29 16:06:52.097	NULL	1
774	AA-10375	Allen Arnold	Consumer	2026-03-29 16:06:52.310	NULL	1
151	AA-10480	Andrew Allen	Consumer	2026-03-29 16:06:51.927	NULL	1
788	AA-10645	Anna Andrews	Consumer	2026-03-29 16:06:52.320	NULL	1
1073	AA-315	Alex Avila	Consumer	2026-03-29 16:06:52.573	NULL	1

The status bar at the bottom indicates 'Query executed successfully.' and 'DESKTOP-CS327V4\MSSQLSERVER... Superstore_DW 00:00:00 Row: 1, Col: 1 9 rows'.

Task 6 — Accumulating Fact Table

Accumulating Fact Table Implementation

The FactSales table was extended with the following additional columns to implement the accumulating fact table pattern:

- `accm_txn_create_time` — Set to the current system datetime (`GETDATE()`) when a record is first inserted into the fact table.
- `accm_txn_complete_time` — Updated when the transaction is completed. Initially NULL until completion data is available.
- `txn_process_time_hours` — A computed column that automatically calculates the difference in hours between `accm_txn_complete_time` and `accm_txn_create_time`.

Package 3: Update_AccumulatingFact.dtsx

```
USE Superstore_DW;
```

```
SELECT TOP 10
```

```
    OrderID,
```

```
    accm_txn_create_time,
```

```
    accm_txn_complete_time,
```

```
    txn_process_time_hours
```

```
FROM FactSales
```

```
WHERE accm_txn_complete_time IS NULL;
```

```
USE Superstore_DW;
```

```
UPDATE FactSales
```

```
SET accm_txn_complete_time = '2026-04-08 18:00:00'
```

```
WHERE OrderID = 'CA-2011-100006';
```

```
USE Superstore_DW;

SELECT

    OrderID,

    accm_txn_create_time,

    accm_txn_complete_time,

    txn_process_time_hours

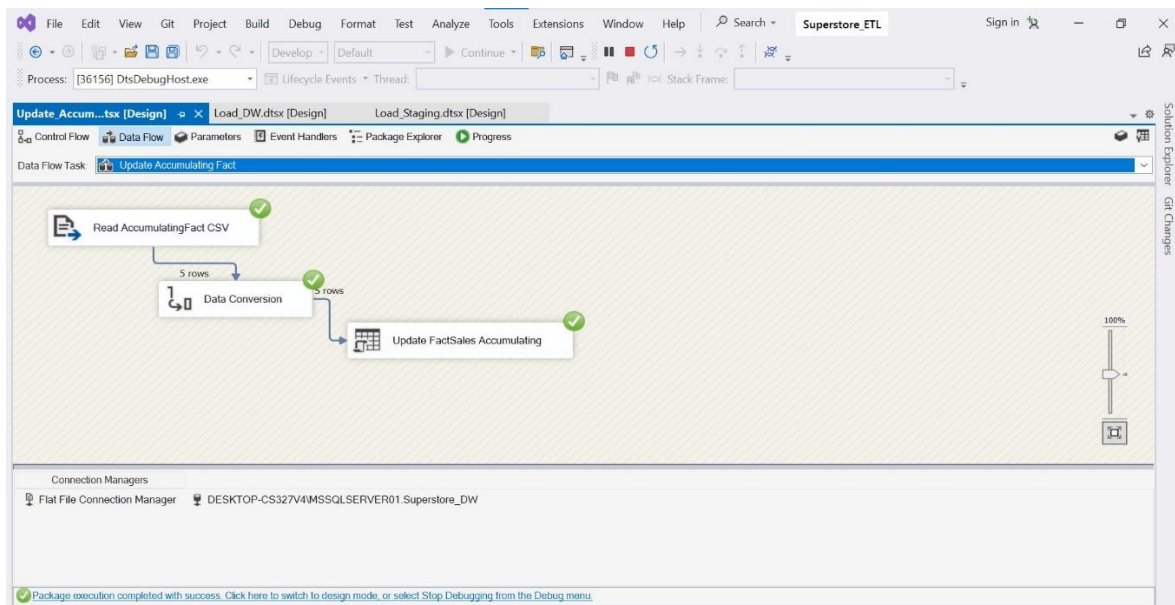
FROM FactSales

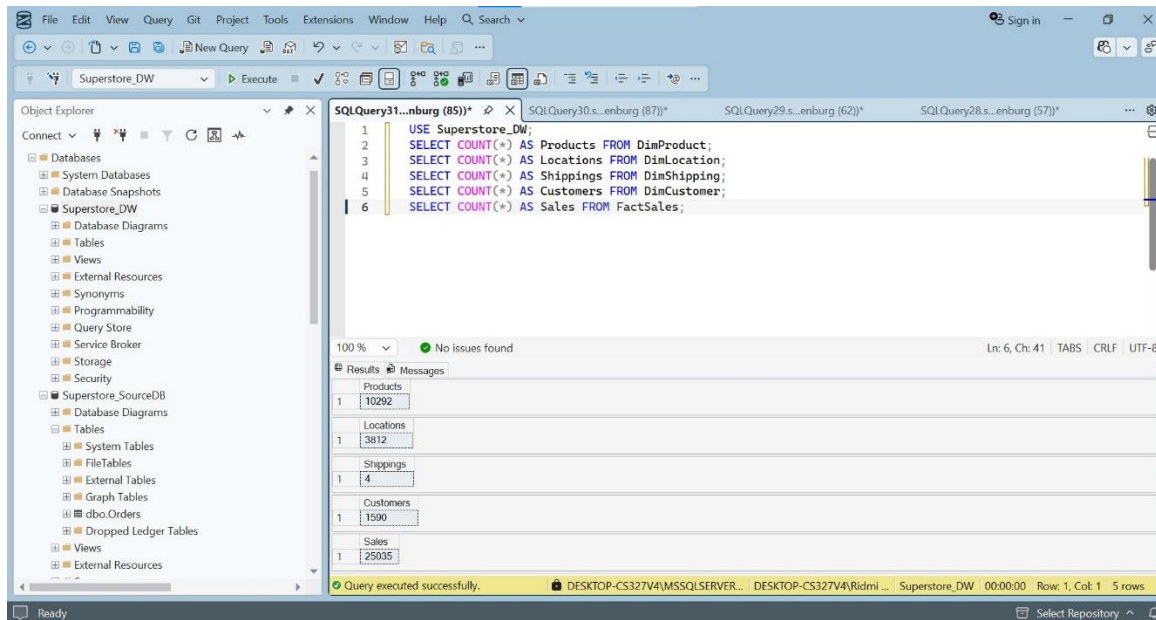
WHERE OrderID = 'CA-2011-100006';
```

A separate SSIS package was created to update the accumulating fact columns. The package reads from a CSV file (AccumulatingFact.csv) containing OrderID and accm_txn_complete_time for completed transactions.

The package uses a Flat File Source to read the CSV, a Data Conversion transformation to handle unicode conversion, and an OLE DB Command to update the corresponding FactSales records with the completion timestamp. The txn_process_time_hours computed column automatically recalculates when accm_txn_complete_time is updated.

A Conditional Split was used to handle NULL values — only rows with a valid accm_txn_complete_time are processed.





Query 12 — Accumulating Fact Update

sql

```
UPDATE FactSales
```

```
SET accm_txn_complete_time = TRY_CAST(? AS DATETIME)
```

```
WHERE OrderID = ?
```

Description: This SQL statement is used in the SSIS OLE DB Command to update the `accm_txn_complete_time` for completed transactions. The `txn_process_time_hours` computed column automatically recalculates.

Query 13 — Verify Accumulating Fact

sql

```
USE Superstore_DW;
```

```
SELECT OrderID,
```

```
    accm_txn_create_time,
```

```
    accm_txn_complete_time,
```

```
    txn_process_time_hours
```

```
FROM FactSales
```

```
WHERE accm_txn_complete_time IS NOT NULL;
```

Description: This query verifies that the accumulating fact columns were successfully updated showing the create time, complete time and calculated process time in hours.